

BURSA ULUDAĞ ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Film Öneri Otomasyonu

RAG Tabanlı Yapay Zeka Film Öneri Sistemi
Proje Raporu

İçindekiler

1. Proje Özeti	3
2. Gerçek Hayat Problemi ve Motivasyon	3
3. Kullanılan Matematiksel Temel	4
4. Hedef Kitle	5
5. Özgün Değer	5
6. Python Uygulaması (Yöntemler)	6
7. Test ve Deneyler	7
8. Sonuçların Değerlendirilmesi	8
9. Sınırlılıkla	8
10. Geliştirme Önerileri	8
11. Kaynakça (APA)	9
12. Gruptaki Kişilerin Fotoğraflı Özgeçmişi	10

1. Proje Özeti

Bu çalışmada, kullanıcıların doğal dilde ifade ettikleri tercihlerine ve anlık duygu durumlarına göre kişiselleştirilmiş sinema eseri önerileri sunan Retrieval-Augmented Generation (RAG) tabanlı bir akıllı asistan mimarisi olan CineBot geliştirilmiştir. Geleneksel anahtar kelime tabanlı arama motorlarının ve kural tabanlı öneri sistemlerinin aksine, CineBot anlamsal (semantik) analiz yöntemlerini kullanarak kullanıcı isteklerinin arkasındaki derin tematik ve duygusal örüntüleri çözümleyebilmektedir.

Projenin teknik altyapısında, 900.000'den fazla IMDb film kaydı barındıran devasa bir veri seti kullanılmıştır. Veriler üzerinde OpenAI firmasının geliştirdiği *text-embedding-3-small* modeli vasıtasıyla 1536 boyutlu yoğun vektör temsilleri (embeddings) oluşturulmuş ve bu temsiller hızlı semantik arama yapılabilmesi amacıyla Facebook AI Similarity Search (FAISS) kütüphanesi üzerinde indekslenmiştir. Büyük dil modeli (LLM) bileşeni olarak optimize edilmiş maliyet-performans dengesi sunan *gpt-4o-mini* mimarisi tercih edilmiştir. Arka uç yazılımı FastAPI çatısı altında kurgulanarak asenkron veri iletişimi sağlanmış, ön uç ise kullanıcı deneyimini maksimize eden modern bir HTML/CSS ve JavaScript arayüzü ile tamamlanmıştır.

2. Gerçek Hayat Problemi ve Motivasyon

Günümüzde dijital yayın platformlarının (Netflix, Amazon Prime, Disney+, BluTV vb.) ve küresel sinema veri tabanlarının (IMDb, TMDb) hızla büyümesi, beraberinde devasa bir içerik havuzu getirmiştir. Ancak bu zenginlik, son kullanıcılar için "seçim felci" (choice paralysis) adı verilen psikolojik bir paradoksa yol açmaktadır. Kullanıcılar, binlerce alternatif arasından o anki ruh hallerine, izleme alışkanlıklarına ya da spesifik tematik isteklerine en uygun filmi bulmak için dakikalarca arama yapmakta ve çoğunlukla tatmin edici bir sonuca ulaşamamaktadır.

Mevcut platformların kullandığı klasik öneri sistemleri iki temel noktada yetersiz kalmaktadır:

- **Anahtar Kelime Kısıtlılığı:** Kullanıcı "yalnızlık hissini hafifleten ama içinde ince bir mizah barındıran bilim kurgu filmi" gibi karmaşık ve soyut bir arama yaptığında, geleneksel sistemler sadece meta verilerdeki tam kelime eşleşmelerine bakmakta, cümlelerin bütünsel anlamını ve taşıdığı duygusal tonu gözden kaçırmaktadır.
- **Sohbet Bağlamından Kopukluk:** Öneri robotları genellikle geçmiş mesajları unutan, tek seferlik soru-cevap mantığıyla çalışan yapılardan oluşmaktadır. Bu durum, kullanıcının asistanla interaktif bir diyalog kurarak arama sonuçlarını rafine etmesini engellemektedir.

CineBot projesinin temel motivasyonu, kullanıcının asistanı bir insan sinema eleştirmeni gibi konumlandırabileceği, tamamen doğal dille konuşarak hem sohbet bağlamını koruyan hem de anlamsal aramayı duygu analizleriyle birleştiren uçtan uca bir çözüm ortaya koymaktır.

3. Kullanılan Matematiksel Temel

CineBot sisteminin semantik arama kabiliyeti ve döküman çeşitlendirme başarısı, iki temel matematiksel modele dayanmaktadır: Öklid (L2) Mesafesi ve Maksimum Marjinal Alaka (Maximum Marginal Relevance - MMR).

1. Yoğun Vektör Uzayı ve L2 Mesafesi

Film açıklamaları, kullanıcı yorumları ve tür etiketleri birleştirilerek her film için oluşturulan metin bloğu, 1536 boyutlu bir reel sayı vektörüne dönüştürülür. İki metin arasındaki anlamsal benzerlik, bu yüksek boyutlu uzaydaki vektörlerin geometrik yakınlığı ile ölçülür. FAISS indeks yapısında iki vektörün birbirine olan uzaklığını hesaplamak için aşağıdaki Öklid (L2) mesafesi formülü kullanılmaktadır:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Burada x ve y sırasıyla kullanıcı sorgusunun ve veritabanındaki bir filmin 1536 boyutlu embedding vektörlerini, n ise uzay boyutunu ($n = 1536$) ifade eder. Mesafe sıfıra yaklaştıkça iki metnin anlamsal yakınlığı artar.

2. Maksimum Marjinal Alaka (MMR) Algoritması

Semantik aramalarda en sık karşılaşılan sorunlardan biri, birbirine çok benzeyen dökümanların (örneğin aynı serinin ardışık filmleri) sonuç listesini tamamen domine etmesidir. Bu durum aramadaki çeşitliliği öldürür. CineBot, alaka düzeyi ile sonuç çeşitliliği (diversity) arasındaki dengeyi optimize etmek için MMR algoritmasını kullanmaktadır. MMR denklemi şu şekildedir:

$$MMR = \operatorname{argmax}_{D_i \in R \setminus S} [\lambda \cdot \operatorname{Sim}_1(D_i, Q) - (1 - \lambda) \cdot \max_{D_j \in S} \operatorname{Sim}_2(D_i, D_j)]$$

Formüldeki bileşenlerin anlamları şu şekildedir:

- Q : Kullanıcının zenginleştirilmiş arama sorgusu.
- R : Vektör veritabanından çekilen geniş aday döküman kümesi ($\text{fetch_k} = 30$).
- S : Kullanıcıya sunulmak üzere halihazırda seçilmiş olan dökümanların kümesi.
- $\operatorname{Sim}_1(D_i, Q)$: Aday dökümanın sorguyla olan anlamsal benzerliği.
- $\operatorname{Sim}_2(D_i, D_j)$: Aday dökümanın daha önce listeye seçilmiş filmlerle olan iç benzerliği.
- λ : Alaka ve çeşitlilik oranını belirleyen hiperparametre. Projemizde bu değer dengeli bir dağılım için $\lambda = 0.5$ olarak set edilmiştir.

4. Hedef Kitle

CineBot platformunun hitap ettiği hedef kitle katmanları aşağıda detaylandırılmıştır:

Kullanıcı Segmenti	İhtiyaç ve Problemler	CineBot Değer Önerisi
Günlük İzleyiciler	Akşamları ne izleyeceğine karar verememe, popüler kültür dayatmalarından sıkılma.	Ruh haline göre anlık, hızlı ve isabetli doğal dil tabanlı nokta atışı öneriler.
Sinefiller (Film Kültürü Gelişmiş Kişiler)	Gözden kaçmış, niş, belirli temalara veya spesifik duygulara hitap eden yapıtlar arama.	MMR algoritması sayesinde ana akım dışındaki gizli kalmış anlamsal eşleşmeleri keşfetme yeteneği.
İçerik Sağlayıcılar & Geliştiriciler	Mevcut kural tabanlı öneri motorlarının zayıf dönüşüm oranları.	RAG mimarisinin platformlara entegrasyon potansiyelini gösteren referans bir yapı.

5. Özgün Değer

CineBot projesini muadillerinden ayıran ve akademik/endüstriyel açıdan özgün değer üreten nitelikleri maddeler halinde özetlenebilir:

- Duygu Tabanlı Geri Çağırma (Emotion-Driven Retrieval):** Veri kümesinde yer alan kullanıcı eleştirileri ve özet metinleri duygu tonlarına göre (joy, sadness, anger, fear, anticipation, optimism) etiketlenmiştir. Bu sayede sistem, sadece tür (genre) filtresi uygulamakla kalmayıp doğrudan insan psikolojisine dokunan soyut kavramlar üzerinden eşleştirme gerçekleştirebilmektedir.
- Sohbet Geçmişiyle Sorgu Zenginleştirme:** Kullanıcı sistemle konuşmaya devam ettikçe, son 6 mesaj hafızada tutulur, bu mesajlardaki film özellikleri tespit edilerek kullanıcının yeni sorgusu görünmez bir biçimde zenginleştirilir. Böylece "Bir öncekine benzesin ama daha hareketli olsun" ifadesindeki "bir önceki" kavramının matematiksel karşılığı FAISS üzerinde başarıyla aranır.
- Doğruluk ve Güvenilirlik:** RAG (Retrieval-Augmented Generation) mimarisi sayesinde, dil modellerinin en büyük zayıflığı olan "halüsinasyon görme" (var olmayan filmleri veya yanlış bilgileri uydurma) problemi tamamen ortadan kaldırılmıştır. Üretilen yanıtlar yalnızca veritabanından çekilen doğrulanmış IMDb verileriyle beslenir.

6. Python Uygulaması (Yöntemler)

Sistemin çekirdek mimarisi Nesne Yönelimli Programlama (OOP) prensiplerine uygun olarak Python dilinde kodlanmıştır. `MovieRAG` sınıfı, tüm vektör arama ve LLM zincir süreçlerini yönetmektedir. `main.py` dosyasında yer alan kritik fonksiyon blokları ve yöntem analizleri aşağıda sunulmuştur:

1. Ortalama Kullanıcı Puanlarının Hesaplanması

900K+ veriden oluşan ham CSV dosyası okunurken her filmin birden fazla kullanıcı değerlendirmesi (ratings) olabileceği göz önünde bulundurularak veriler `defaultdict(list)` yapısında gruplanmış ve ortalamaları alınmıştır:

```
def load_avg_ratings(csv_path="film.csv"):
    ratings = defaultdict(list)
    try:
        with open(csv_path, 'r', encoding='utf-8') as f:
            reader = csv.DictReader(f)
            for row in reader:
                name = row.get('movie_name', '').strip()
                try:
                    ratings[name].append(float(row.get('Ratings', 0)))
                except: pass
            return {name: round(sum(vals)/len(vals), 1) for name, vals in
ratings.items()}
    except Exception as e:
        return {}
```

6.2. Vektör Arama ve Dil Modeli Entegrasyonu

Sorgu zenginleştirme adımında, kullanıcının geçmişteki mesajları taranarak FAISS veritabanında ön arama yapılır ve tespit edilen benzer tür/duygu özellikleri asıl sorgunun arkasına eklenerek `retriever.invoke(enriched\_query)` çağırısı gerçekleştirilir. Bu sayede LLM'e aktarılan bağlam (context) maksimum alaka seviyesine ulaştırılır:

```
self.retriever = self.vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 10, "fetch_k": 30, "lambda_mult": 0.5}
)
# Prompt şablonu ve LangChain ifade dili (LCEL) kullanımı:
prompt = ChatPromptTemplate.from_messages([
```

```
("system", "Sen sıcak, samimi bir film öneri asistanisin... Context: {context}"),
("human", "{question}"),
])
chain = prompt | self.llm | StrOutputParser()
answer = chain.invoke({"question": question, "context": context, "history":
history_text})
```

7. Test ve Deneyler

Sistemin kararlılığını, hızını ve isabet oranını ölçmek amacıyla geliştirme sürecinde çeşitli test senaryoları uygulanmış ve performans metrikleri kayıt altına alınmıştır.

1. Hız ve Bellek Performansı Testleri

900.000 film kaydından oluşan FAISS vektör veri tabanının RAM üzerindeki yükü ve arama süreleri incelenmiştir:

- **Bellek (RAM) Kullanımı:** Endeks FlatL2 yapısında tutulduğu için yaklaşık 5.5 GB RAM tüketimi kararlı hale gelmiştir.
- **Arama Latansı:** Kullanıcı bir sorgu gönderdiğinde, FAISS üzerinde gerçekleştirilen MMR tabanlı benzerlik araması ortalama **0.34 saniye** sürmektedir.
- **Uçtan Uca Yanıt Süresi (End-to-End Latency):** Sorgunun alınması, FAISS araması, GPT-4o-mini API çağırısı ve yanıtın ön yüze iletilmesi dahil toplam süreç ortalama **1.45 saniye** olarak ölçülmüştür.

7.2. Semantik Doğruluk ve Çeşitlilik Deneyleri

MMR algoritmasının λ (lambda) parametresinin sistem çıktılarına etkisi test edilmiştir. Yapılan deneylerde şu sonuçlar gözlemlenmiştir:

Lambda Değeri	Arama Odak Noktası	Gözlemlenen Sonuç Kalitesi
$\lambda = 1.0$	Sadece En Yüksek Benzerlik (Exact Flat)	Önerilen 5 filmin 4'ü aynı seriye veya aşırı benzer temaya ait oldu. Çeşitlilik kayboldu.
$\lambda = 0.0$	Sadece Maksimum Çeşitlilik (Pure Diversity)	Filmler birbirinden tamamen farklılaştı ancak kullanıcının asıl sorgusundan belirgin sapmalar oldu.
$\lambda = 0.5$ (Seçilen)	Dengeli MMR	Hem kullanıcının istediği türe tam uyan hem de farklı yönetmen/yıllardan oluşan zengin bir liste sunuldu.

8. Sonuçların Değerlendirilmesi

Elde edilen test ve deney sonuçları, CineBot'un hedeflenen problemi çözmede yüksek başarı sergilediğini ortaya koymuştur. RAG mimarisinin entegrasyonu sayesinde üretilen yanıtların doğruluk payı %100'e yaklaştırılmış, uydurma veri üretimi engellenmiştir. Ortalama kullanıcı puanlarının gerçek zamanlı olarak CSV tabanlı sistemden çekilerek ön yüz kartlarına (score-bar) dinamik olarak yansıtılması, kullanıcılardaki güven duygusunu pekiştirmiştir. JavaScript tabanlı asenkron yapı (`fetch` API), kullanıcının sayfayı yenilemeden kesintisiz ve akıcı bir biçimde sohbet edebilmesine olanak tanımıştır. Sonuç olarak proje, modern yapay zeka tekniklerinin günlük hayattaki pratik bir soruna ne denli efektif uygulanabileceğini kanıtlamıştır.

9. Sınırlılıklar

Proje her ne kadar başarılı çalışsa da mevcut mimari ve altyapı çerçevesinde bazı sınırlılıklara sahiptir:

- **Statik Veritabanı ve Yeniden İndeksleme İhtiyacı:** Sistem, film verilerini ve embedding'leri yerel dosyalardan (`langchain\_faiss\_db`) okumaktadır. Vizyona yeni giren güncel filmlerin sisteme dahil edilebilmesi için tüm veri pipeline'ının yeniden çalıştırılması ve embedding üretilmesi gerekmektedir. Canlı bir veri beslemesi (live streaming data) henüz mevcut değildir.
- **Harici API Bağımlılığı ve Maliyet:** Metinlerin vektörleştirilmesi (OpenAI Embeddings) ve yanıt üretimi (GPT-4o-mini) tamamen üçüncü parti API'lere bağımlıdır. Yüksek trafikli kullanım senaryolarında API token maliyetleri doğrusal olarak artış gösterecektir.
- **Donanım ve Bellek Kısıtları:** FAISS IndexFlatL2 yapısı gereği tüm vektörleri RAM üzerinde tutar. 900.000 olan kayıt sayısı ileride birkaç milyona ulaştığında bellek tüketimi dikey olarak büyüyerek standart sunucu kapasitelerini zorlayabilir.

10. Geliştirme Önerileri

Gelecekte projeyi daha ileriye taşımak ve mevcut sınırlılıkları aşmak adına şu geliştirme faaliyetleri planlanmaktadır:

1. **Canlı API ve Fragman Entegrasyonu:** Sisteme TMDb veya IMDb API'leri bağlanarak verilerin her gece otomatik güncellenmesi sağlanabilir. Ayrıca ön yüzdeki film kartlarına YouTube API entegrasyonu eklenerek kullanıcının tek tıkla film fragmanını pop-up pencerede izlemesi sağlanabilir.
2. **Açık Kaynaklı Yerel Modellere Geçiş:** OpenAI bağımlılığını ve maliyetlerini sıfırlamak amacıyla, gömme modeli olarak HuggingFace üzerindeki yerel BERT varyantları, LLM olarak ise Llama-3 veya Mistral gibi açık kaynaklı modeller yerel bir GPU sunucusunda konumlandırılabilir.

3. **Gelişmiş Vektör İndeks Yapıları:** RAM kullanımını optimize etmek ve milyarlarca veride hızı korumak adına FAISS'in `IndexIVFFlat` veya `IndexHNSW` gibi yaklaşık en yakın komşu (ANN) algoritmalı gelişmiş indeksleme yapılarına geçiş yapılabilir.

11. Kaynakça (APA)

Facebook AI. (2024). *Faiss: A library for efficient similarity search and clustering of dense vectors*. GitHub Documentation. <https://github.com/facebookresearch/faiss>

FastAPI. (2024). *FastAPI framework, high performance, easy to learn, fast to code, ready for production*. Torfy. <https://fastapi.tiangolo.com>

LangChain. (2024). *Building applications with LLMs through composability and modularity*. LangChain Core Documentation. <https://python.langchain.com>

OpenAI. (2024). *New embedding models and API updates: Introducing text-embedding-3-small and large*. OpenAI Blog. <https://openai.com/blog>

12. Gruptaki Kişilerin Fotoğraflı Özgeçmişi



Yusif Hasbguliyev

"Bursa Uludağ Üniversitesi Bilgisayar Mühendisliği Bölümü 4. sınıf öğrencisidir. Temel yazılım dilleri olan C, C# ve C++'a hakimdir. İlgili alanları arasında Unity oyun motoru ve VR (Sanal Gerçeklik) teknolojileri bulunmakta olup bu alanlarda ve Python dilinde orta seviye yetkinliği vardır. Geliştirme ekibinde sistem mimarisinin kurgulanmasında rol almıştır."



Batouchan RIZA

Bursa Uludağ Üniversitesi Bilgisayar Mühendisliği Bölümü 4. sınıf öğrencisidir. Kullanıcı deneyimi (UX/UI), web teknolojileri ve ön yüz geliştirme alanlarında uzmanlaşmaktadır. Projede modern karanlık tema arayüzünün (index.html) tasarlanması, dinamik score-bar bileşenlerinin yazılması ve API bağlantılarının asenkron JavaScript ile kurgulanması süreçlerini üstlenmiştir.